

OBJECT ORIENTED ANALYSIS AND DESIGN

Objects mean that they can have their own data. It is an instance of a class. It can be your name, date of birth, password, etc. These are all objects.

Class is a collection of objects. A student account should have a roll number, student name, etc. It should be relevant data.

Abstraction defines the character of any particular object, or only the relevant information that is required for a task. It hides unnecessary implementation details and shows only essential features.

Encapsulation is protection of your data. It checks whether variables are declared as public or private, and allows data to be accessed accordingly. For instance, an ATM checks if a card is valid, and a PIN entered is correct, before it allows you to withdraw cash.

Polymorphism is the ability of a single interface—such as a variable, function, or operator—to represent or handle multiple different forms. The same name or 'button' can be used for different tasks, depending on the data it's given, or the object it belongs to.

- **Subtype Polymorphism** is a type of polymorphism that is dynamic, where a single method is defined in a base class and the program lets child classes provide their own version only during runtime (e.g. A draw() program doesn't know whether to draw a rectangle or circle until it sees the identity of the object).
- **Function overloading (ad-hoc polymorphism)** is a type of polymorphism, where a function may have the same name but works in different ways when the function is being called, because it is being provided with different data. (e.g. A program for addition can take integers, as well as floating numbers).

Inheritance means that another class or sub-class can inherit the functionality of the class being called. It's called a parent (or base) class, and the other one is called a child (or derived) class.

Why is OOP important?

- *Programs and modules can be re-used.* Otherwise, you won't remember what functionality you used. Rate of interest, for instance, cannot be rewritten for different age ranges. Instead, a general logic is written that can allow for variables (eg. age) to be inserted so that the rate can be calculated.
- *It models real-world problems.* Objects are used to represent things (like a bank account), inheritance helps classify the thing (e.g. dog as animal, or car as vehicle), abstraction hides unnecessary details that people in real life often do not know (or need to know, you can drive a car without knowing how the engine works), etc.
- *It allows for easy maintenance.* If you want to change the logic in the future for rate of interest, with OOP, if you change it in one place, it's changed everywhere.

- *Applications become scalable.* You can add features without rewriting/breaking old ones, and can localize changes because every system is organized into classes, modules, components, etc. with its own independent responsibilities.
- Acts as a foundation for Java, C+, Python, C#.

UML DIAGRAMS

UML means Unified Modelling Language. It is a standard way to visualize, design, and document software systems (especially object-oriented ones) and data in an easy-to-understand manner with the use of diagrams.

Types of Diagrams:

1. **Class Diagrams** explain classes and their relationships to each other.
2. **Object Diagrams** explain objects and their relationships to each other.
3. **Use-case Diagram** shows how an action occurs and how a program's logic behaves.
4. **Sequence Diagram** shows the exact order (and at what time) something occurs in the code. Things don't happen at the same time, the next step happens based on your input.
5. **Collaboration Diagram** shows how multiple objects interact with each other in the program.
6. **State-chart Diagram** is similar to a flowchart, but instead of showing procedures and steps, it shows an object's behavior by showing its different states and the events that cause transitions between them. (Paid → Shipped → Delivered are states triggered by events, and the arrows are transitions between events)
7. **Activity Diagram** shows what happens in a function and how it works.

Class Diagram consists of Analysis Class Diagram and Design Class Diagram, either of which can be used. Analysis does not mention data types, but Design does. This is the only difference.

Use-case Diagram has:

- Actors (People involved in the situation, or other systems, which exist outside the system boundary.)
- Use-case (Things that an actor can do, like log-in or withdraw cash.)
- Relationship between actor & use-case (Shows who can do what. Customer withdraws cash, bank system authenticates user, etc.)
- Relationship between use-cases (If one use-case depends on another one. Cash withdrawal relies on user being authenticated.)
- Relationship between actors (Usually related to inheritance, and not an actual interaction between two different actors, especially if not interacting with the system. Premium customers can do everything that customers can, with a few additions.)
- System boundary (What is inside and outside the system. Everything that the ATM does is inside the box, and any person or system that interacts with it is outside.)

Sequence Diagram has life-lines, and messages. First you enter your PIN, then you're able to check your bank balance, etc.

DEFECT LIFE CYCLE

Errors, defects and bugs are very similar and may be used interchangeably, but there are key differences between them.

An error can be a mistake that occurs when the developer fails to understand a requirement definition, and that misunderstanding gets translated into buggy code. They are generated due to wrong logic, syntax, or loop that can impact the end-user experience. They can lead to bugs or defects.

A defect can be a logical error, and it occurs when the software does not meet the requirements or specifications outlined during the SDLC. For instance, the rate of interest should be calculated for 365 days, but in the program it's 300 days.

A bug is an issue in the software that causes it to behave unexpectedly. They can be caused by incorrect logic, poor design, or even a mistake made during the coding process.

- **Step 1:** A new bug is found and it is added and labelled as 'open' in Jira by the tester.
- **Step 2:** The developer works on the bug and fixes it. It is labelled as 'in progress' when the developer is working on it, then changed to 'waiting for QA' or 'pending retest' when the developer is done.
- **Step 3:** Retesting is done by the QA team to ensure the fix works as expected, and that nothing else in the program is impacted by the fix. It is labelled as 'in QA' or 'retesting'. If it is still buggy, it is reopened and handed to a developer to fix. This part will loop until it is completely fixed and tested.
- **Step 4:** Once everything is complete and verified, the bug is labelled as 'closed' in Jira.

If there are 20 bugs to be fixed, they need to be prioritized since they cannot all be fixed at once, and chances are, they don't all need to be fixed. They can be prioritized based on impact and what bugs affect the code the most, but they can also be prioritized based on what is important for the customer and what needs to be delivered to them.

Severity is the impact of the bug in the application and how much it prevents it from working properly. **Priority** is how important the bug is based on preference.

Severity has 4 categories:

- **Critical:** Needs to be fixed immediately. For instance, the login feature does not work.
- **High:** When there are no critical bugs, take care of high-severity bugs. For instance, a specific payment method (e.g. debit card) fails, but credit cards work.

- **Medium:** If no high-severity, take care of these. For instance, cannot search past orders older than a year.
- **Low:** If no medium-severity, take care of these. For instance, incorrect colors or small visual glitches.

Testers should always include screenshots of the bug they found as evidence that the bug has actually occurred, and to help explain the issue. This ensures a lack of ambiguity.

[Bug Testing Sheet Example](#)

If a requirement changes and it impacts many things within the program, it may be difficult to know what exactly needs to be changed, and what is connected to the change. **RTM** helps us understand what is pending and what fields need to be modified because of a change. However, sometimes an RTM isn't necessary, like if you're using Jira. If you have an Epic that changes, and there are established Tasks and Stories within it, you know what is connected to the Epic and what needs to be changed too.

RTM Sheet Columns: Requirement number (1.1, 1.2), Requirement Description, Scenario number, Test case number, Test case result/status (pass/fail/work in progress/open)

Forward Traceability means it starts from the requirements and ends at the test cases.

Backward/Reverse Traceability means it starts from test cases and ends at the requirements.

Smoke Testing should ensure that there is nothing critical that can stop the testing prematurely (or, that the build contains no 'showstoppers'). If something breaks, you cannot test further. Or, if you cannot log in, you cannot conduct any further testing either. Usually performed by the development team.

Sanity Testing/Narrow Regression Testing is done by the testers. Also ensures that the application does not contain any showstoppers, and determines if a build is stable and sensible enough to proceed with more extensive testing.

If smoke testing is done by the developers, sanity testing will be skipped, or vice versa. There is no need for both.